



Efficient Workload Prediction Model for Containerized Cloud

Tianqi Xie^{1,2}, Yunfei Zhang^{3(✉)}, Changhong Tai^{1,2}, Tariq Ali Arain²,
and Usama Ali²

¹ Key Laboratory of Water Big Data Technology of Ministry of Water Resources,
Hohai University, Nanjing, China
{xtq2001,220207040007}@hhu.edu.cn

² College of Computer and Software, Hohai University, Nanjing, China

³ College of Computer Science and Software Engineering, Hohai University,
Nanjing, China
yunfeizhang@hhu.edu.cn

Abstract. Containerization is increasingly used in cloud service for its convenience and heterogeneous consistency. The container workload is the combination of the coexistence of second-level burst requests and hour-level periodic tasks, and that makes the workload prediction a complex and time-consuming job. However, container instance requires rapid prediction method to ensure the minute-level elastic scaling, thus make a conflict. In this paper, we propose an efficient neural network model for workload prediction. Our approach combines flash-attention mechanism with Long Short-Term Memory neural network to improve the efficiency and performance of our model on the real-world workload data from the Alibaba Cluster Trace dataset. Finally, we compare the training time and performance of our model with the LSTM-based model and the Transformer-based model in the different time scales, and it brings up to 15.18% improvement in performance and an average of nearly 20% less training time. The result of the experiment demonstrates its efficiency and better performance.

Keywords: Cloud Service · Containerization · Workload Prediction · Deep Neural Network

1 Introduction

Workload prediction and optimization are critical for Cloud Service Providers (CSPs). The evolution from Infrastructure as a Service (IaaS) to Function as a Service (FaaS), coupled with advances in virtualization architectures, enables efficient and scalable application deployment across diverse environments [1, 2] like microservice architectures and edge computing. Containerization packages applications and dependencies into standardized images [3, 4] ensuring consistent execution. Real-time monitoring and predictive models allow CSPs to anticipate workload, enabling proactive resource scaling and migration to ensure QoS.

Existing research [5,6] relies on single-pattern assumptions with mathematical, statistical, or machine learning methods. Real-world environments show multiscale load patterns [7,8] with short-term bursts, periodic tasks, and complex resource dependencies. Container instances require minute-level elastic scaling, but traditional mechanisms have low training efficiency, delaying scaling responses in production.

In this paper, we review existing research and propose neural network-based innovations to address cloud container load prediction challenges in real-world production environments. The primary contributions of this paper are summarized as follows:

- We integrate attention mechanisms into baseline models to address varied workload patterns across time scales, improving contextual dependency capture while maintaining efficiency.
- We combine the flash attention mechanism [9] with LSTM neural network to enable rapid training without compromising prediction accuracy. The attention mechanism improves LSTM performance, requires less data, and improves training efficiency.
- We evaluate our method using real-world workload traces from Alibaba Cloud Center [10], comparing the prediction accuracy(MAE) and execution time in hour-level, minute-level and second-level.

2 Related Work

Existing research on container instance workload prediction is broadly categorized into three methodological paradigms: statistical-based methods, machine learning (ML)-based methods, and deep neural network (DNN)-based methods.

Statistical-based methods, such as the Exponentially Weighted Moving-Average (EWMA) model [5] and Autoregressive Integrated Moving Average (ARIMA) model [8,11], leverage mathematical and statistical models to emulate periodic trends or stationary workload patterns. Some researchers found the difference between different patterns and combine them with a heuristic model [12]. These approaches are well-suited for periodic workload environments but struggle to accommodate rapidly fluctuating burst workloads.

ML-based methods, including Support Vector Machine (SVM) [7] and Bayesian Ridge Regression (BRR) [13], employ supervised learning to infer non-linear mappings between historical data and future workload, offering improved adaptability to burst workloads. It has the ability to exhibit superior adaptability to burst workloads, but their optimal performance remains restricted to environments with periodic workload patterns.

DNN-based prediction methods utilize deep neural network like Convolutional Neural Networks (CNN) [14], Long Short-Term Memory (LSTM) [6], BiLSTM [15], and Transformer [16] to autonomously capture intricate temporal dependencies, spatial correlations, and multiscale features in dynamic cloud environments, thereby addressing limitations of traditional methods in handling complex, non-stationary workload dynamics. However, the ability to capture contextual dependencies in workload patterns varies between neural architectures due to structural differences, and few of them realize the existence of the

coexistence of second-level burst requests and hour-level periodic tasks, which result in the limitation in its performance.

3 Problem Description and Methodology

3.1 Problem Description

This paper focuses on the prediction of workload in containerized cloud computing environments, leveraging historical trace data to forecast future server workload states. Monitoring metrics, such as CPU, memory, and network bandwidth [17], is vital for efficient resource allocation and cost optimization, requiring systematic control. Existing research highlights the CPU and memory utilization as key workload indicators [18, 19], thus chosen as our model's test targets.

Current studies based on real-world datasets from Alibaba Cloud [10] and Google Cloud [20] identify CPU and memory utilization as key bottlenecks causing workload imbalance and reduced server performance. Analysis [21, 22] reveals memory utilization on Alibaba Clusters reaches 80%, with CPU utilization below 40%, indicating heavy memory use and poor CPU resource allocation. Optimizing CPU utilization via systematic strategies can boost performance, reliability, and cost efficiency by minimizing excess infrastructure costs.

3.2 Methodology

Bao et al. [23] demonstrated that the workload patterns of real-world service clusters can be categorized into four distinct types on arbitrary temporal scales: monotonous increase, monotonous decrease, increase-decrease and decrease-increase. As lightweight deployment units of cluster servers, container instances exhibit workload characteristics that retain morphological similarity to the macroscopic load patterns of their host servers while operating at a reduced scale. Building upon this framework, we adopt their methodology [23] to construct a container load prediction model through a three-phase workflow: (1) data preprocessing, (2) model training, and (3) workload prediction, with the aim of addressing the resource load prediction problem for containerized systems. The schematic of this methodology is illustrated in Fig. 1.

- Data preprocessing. Firstly, a sliding window sampling technique is employed on the raw workload data to create the workload sequence. To normalize the container attribute parameters and workload data extracted from the raw dataset, the Z-score normalization method is first applied to the original data in normalized values. This process facilitates data processing and accelerates model convergence. Subsequently, a sliding window sampling technique is used in the time series to construct a structured dataset, which is partitioned into training and testing sets in a 7: 3 ratio.
- Model training. The model training phase comprises two components: clustering and training. We separate the raw sequence data into 4 different datasets using clustering method. Each dataset represents one workload pattern. The

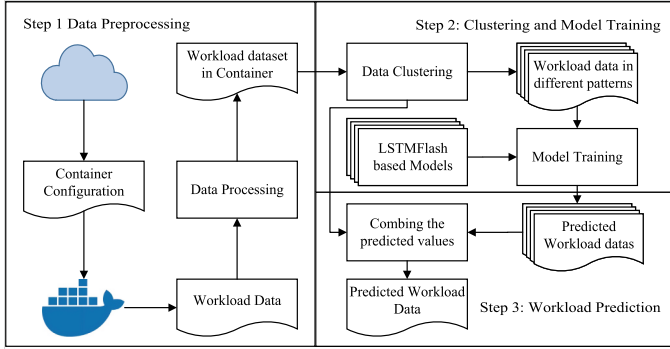


Fig. 1. Methodology Figure

models are independently trained on their respective cluster subsets using neural networks, using mean absolute error to calculate the performance of the model.

- **Workload prediction.** The distance between the input workload sequence and each cluster centroid is computed to generate a weight vector. Each model produces a predicted workload value based on the input sequence. The final predicted load value is derived by aggregating these outputs through a weighted summation guided by the weight vector.

Data Preprocessing. To address the container and its workload prediction problem, we define container workload as its continuous CPU or memory utilization $x_t (t \in [0, n])$, which denotes the utilization rate of the CPU or memory workload at time t . The data sequence X is constructed using a sliding window, defined as:

$$X_k = [x_{k\tau}, x_{k\tau+1}, \dots, x_{k\tau+T-1}] \quad (1)$$

where k denotes the window index, τ denotes the step size, T denotes the window length.

For the historical workload sequence X_k , we apply Z-score normalization to the raw data into standardized values U_k within $[0, 1]$. This transformation can make data more convenient for processing and improves convergence speed while training. The normalization is calculated as follows:

$$Z = \frac{x - \mu}{\sigma} \quad (2)$$

where μ denotes the mean of the raw data u , σ represents its standard deviation.

We get the workload data $U = [u_1, u_2, \dots, u_n]$, $u_i \in [0, 1], i \in [0, n]$ after normalization, u_i denotes the utilization rate of the resource, which represents the workload. After that, we divide the training data and the testing data in the 7: 3 ratio.

Clustering and Model Training. The normalized sequential dataset, which includes comprehensive container workload data, is processed using the BIRCH clustering algorithm [24] to identify the cluster approach. For each sample x_i in sequence $X = [x_1, x_2, \dots, x_n]$, we will calculate its distance to each cluster center $c_j \in C$ and select the top b nearest centroids and add it into its dataset. The details of the clustering and dataset generation are provided in Algorithm 1. For each workload cluster $D = \{d_1, d_2, d_3, d_4\}$, a dedicated neural network model M is trained to predict workload value. We use the normalized sequence U as input value to train M and get the predicted value $\hat{y}$, and evaluate the model by the Mean Absolute Error(MAE) using the following function:

$$MAE(y, \hat{y}) = \frac{1}{dim(\hat{y})} \sum_{t=1}^{dim(\hat{y})} |\hat{y}_t - y_t| \quad (3)$$

where y denotes the true value and $\hat{y}$ denotes the predicted value, $dim(\hat{y})$ denotes the dimension of y .

Algorithm 1. Clustering and Dataset Generation

Require: Workload samples $X = [x_1, x_2, \dots, x_n]$, Cluster number k , Pattern degree b

Ensure: Dataset for each pattern $D = \{D_1, D_2, D_3, D_4\}$

```

1:  $T \leftarrow ConstructCFTree(X)$ 
2:  $C \leftarrow ExtractCentroids(X)$ 
3: for  $i = 1 \rightarrow k$  do
4:    $D_i \leftarrow \emptyset$ 
5: end for
6: for  $x \in X$  do
7:    $d = \langle \rangle$ 
8:   for  $x \in X$  do
9:      $d_i = \sqrt{\|x - c_i\|^2}$ 
10:    append  $d_i$  to  $d$ 
11:   end for
12:   sort  $d$ 
13:   for  $i = 1 \rightarrow b$  do
14:      $j \leftarrow pop(d)$ 
15:      $D_i \leftarrow D_i \cup \{x\}$ 
16:   end for
17: end for
```

a. Baseline model

Bao et al. [23] demonstrated effective model training using LSTM for time series prediction stasks. The LSTM neural network [25], an advanced variant of Recurrent Neural Network (RNN) [26], mitigates the gradient and vanishes of the gradients inherent in traditional RNN by incorporating a cell state propagation mechanism and gated units (input gate, output gate and forget gate) to regulate information flow across long sequences.

For the input workload data x_t and the hidden state in the previous time step h_{t-1} , the input gate computes the update ratio of the current state i_t and generates a candidate cell $\widetilde{C}_t$ as follows:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (4)$$

$$\widetilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad (5)$$

where h_{t-1} denotes the hidden states in the previous time step, x_t denotes the present input value, σ denotes the sigmoid function, W_i and W_c denotes the weights of i and C , b_i and b_c denotes the bias of i and C .

The forget gate regulates the retention ratio of historical information within the cell state. By processing the concatenated vector of the previous hidden state h_{t-1} and the current input x_t , it outputs the ratio $f_t \in [0, 1]$ of the proportion of historical features preserved for a subsequent time step. The f_t is calculated as follows:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (6)$$

where σ denotes the sigmoid function, h_{t-1} denotes the hidden states in the previous time step, x_t denotes the present input value, W_f denotes the weights of f , b_f denotes the bias of f .

The cell state is updated by the retention ratio f_t , the historical state C_{t-1} , the candidate cell state $\widetilde{C}_t$ and the update ratio i_t as follows:

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \widetilde{C}_t \quad (7)$$

The output state calculates the ratio of the cell state o_t propagated to the hidden layer and updates the hidden state h_t as follows:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (8)$$

$$h_t = o_t \cdot \tanh(C_t) \quad (9)$$

where σ denotes the sigmoid function, h_{t-1} denotes the hidden states in the previous time step, x_t denotes the present input value, W_o denotes the weights of f , b_o denotes the bias of f .

According to the hidden state h_t , we calculate the predicted workload value $\hat{y}_t$ using the following equation:

$$\hat{y}_t = W_y \cdot h_t + b_y \quad (10)$$

where o_t denotes the ratio, σ denotes the sigmoid function, h_{t-1} denotes the hidden states in the previous time step, x_t denotes the present input value, W_o denotes the weights of f , b_o denotes the bias of f .

Finally, we use Function 3 to calculate the MAE between the true value y_t and the predicted value $\hat{y}_t$, and we use Adam optimizer to optimize the LSTM model.

b. Transformer-based model

Since the emergence of the Transformer [27], its superior performance in time series prediction has been extensively demonstrated [28]. This effectiveness extends to server and container workload forecasting tasks [29]. Therefore, this paper replaces the LSTM-based baseline model with Transformer-based model to further enhance training performance and predictive accuracy.

For the input workload sequence $X = [x_1, x_2, \dots, x_n]$, the linear layer is firstly applied to the raw data as follows:

$$E = W_E \cdot X + b_E \quad (11)$$

where $W_E \in \mathbb{R}^{d \times d_{model}}$ denotes the weight of E and b_E denotes the bias of E .

Then, positional encoding is applied to the sequence embeddings using the following equation:

$$PE_{(pos, 2i)} = \sin \frac{pos}{10000^{\frac{2i}{d_{model}}}} \quad (12)$$

$$PE_{(pos, 2i+1)} = \cos \frac{pos}{10000^{\frac{2i}{d_{model}}}} \quad (13)$$

And we get the input sequence Z as follows:

$$Z = E + PE, Z \in \mathbb{R}^{n \times d_{model}} \quad (14)$$

After positional encoding, we use the learnable weight W_Q, W_K and W_V to calculate Q, K, V first through linear transformations:

$$Q = ZW_Q, K = ZW_K, V = ZW_V \quad (15)$$

Then, we calculate the attention weights as follows:

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V \quad (16)$$

Concatenate each outputs of all heads and project them:

$$MultiHead(Q, K, V) = Concat(head_1, head_2, \dots, head_h) \cdot W_O \quad (17)$$

where $W_O \in \mathbb{R}^{d_{model} \times d_{model}}$ denotes the weight of multi-head attention outputs

The encoder output is computed through a feed-forward network with residual connections and layer normalization, which is then passed as input to the subsequent encoder layer. Formally, the output Z'' is derived as:

$$Z' = LayerNorm(Z + MultiHead(Q, K, V)) \quad (18)$$

$$FFN(Z') = ReLU(Z'W_1 + b_1)W_2 + b_2 \quad (19)$$

$$Z'' = ReLU(Z' + FFN(Z')) \quad (20)$$

where W_1, W_2 denotes the weight metrics, b_1, b_2 denotes the bias, $LayerNorm(\cdot)$ denotes the layer normalization function and $FFN(\cdot)$ denotes the Feed Forward Function.

We use the last timestep as the model's output after repeating the computing from positional encoding to Z'' for N times:

$$h_{model} = H[-1] \quad (21)$$

Finally, we use a linear layer to get the predicted value with the weight W_y and bias b_y :

$$\hat{y}_t = W_y \cdot h_{final} + b_y \quad (22)$$

c. LSTMFlash-based model

Despite its widespread success, the Transformer neural network [27] often suffers from prohibitive training time demands and suboptimal performance under limited training data [30]. To address the computational efficiency and memory footprint bottlenecks inherent in Transformer training, Dao et al. [9] proposed the Flash-Attention mechanism and its enhanced Transformer. These improvements leverage tiling strategies, kernel fusion, and selective recomputation to optimize memory usage while accelerating attention computation, achieving significant reductions in GPU memory consumption without compromising accuracy.

Traditional multi-head attention mechanism are calculated as a whole, but flash attention separates it into several blocks and calculates them one by one. Q, K, V is divided into B blocks:

$$Q = \begin{bmatrix} Q_1 \\ \dots \\ Q_B \end{bmatrix}, K = \begin{bmatrix} K_1 \\ \dots \\ K_B \end{bmatrix}, V = \begin{bmatrix} V_1 \\ \dots \\ V_B \end{bmatrix} \quad (23)$$

For the rows i and columns j , the local attention O_{ij} is computed as follows:

$$S_{ij} = \frac{Q_i K_j^T}{\sqrt{d}} \quad (24)$$

$$O_{ij} = \text{softmax}(S_{ij}) V_j \quad (25)$$

Flash attention mechanism employs the online softmax with the cumulative maximum tracking to prevent the numerical overflow during block-wise attention computation. It updates the maximum value m and summation terms l across partitioned blocks:

$$m_k = \max(m_{k-1}, \max_j S_{ij}) \quad (26)$$

$$l_k = \lceil^{m_{k-1}-m_k} l_{k-1} + \sum \lceil^{S_{ij}-m_k} \quad (27)$$

Finally, it accumulate all the attention in blocks to obtain the final result m_k .

$$O_k = \frac{e^{m_{k-1}-m_k} l_{k-1}}{l_k} O_{k-1} + \frac{\sum \lceil^{m_{k-1}-m_k}}{l_k} O_{ij} \quad (28)$$

To avoid the problem of unstable training performance of Transformer neural networks when facing small datasets, considering the excellent performance of baseline models in small-scale models with small data volumes, this paper proposes an LSTM neural network combined with flash attention mechanism. The details of the training are provided in Algorithm 2.

Algorithm 2. Training flash attention based LSTM Model

Require: Workload samples $X = [x_1, x_2, \dots, x_n]$, Batch Size bs, Epoch Number N, Number of layers k

Ensure: Predicted value $\hat{y}_t$

```

1: for  $epoch = 1$  to  $N$  do
2:   for  $t = 1$  to  $k$  do
3:      $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$ 
4:      $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$ 
5:      $\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$ 
6:      $C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$ 
7:      $o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$ 
8:      $h_t = o_t \cdot \tanh(C_t)$ 
9:      $Q = ZW_Q, K = ZW_K, V = ZW_V$ 
10:     $h_{attn} = FlashAttention(Q, K, V)$ 
11:     $h_t = h_{LSTM} + h_{attn}$ 
12:     $\hat{y} = W_y \cdot h_{comb} + b_y$ 
13:     $\hat{y}_t =[:, -1 :, :]$ 
14:   end for
15: end for
16: return  $\hat{y}_t$ 

```

Workload Prediction. After the data preprocessing and model training mentioned above, the models corresponding to each mode have learned distinct features of workload patterns. To predict a sequence X , it first evaluate the distance to between X and each cluster center, then dynamically adjusts the weights of each model based on these distances. Finally, it combines the predicted results from different models with corresponding weight to obtain the prediction sequence and output the aggregated result as the final prediction.

4 Performance Analysis

4.1 Experiment Configuration

Experiment Environment. The experimental framework was implemented using PyTorch 1.13.1 on a server equipped with three NVIDIA GeForce RTX 3090 GPUs and two Intel Xeon Gold 6240@2.6GHz processors. All comparative methods were trained and evaluated under identical hardware and software configurations using the same dataset to ensure fairness.

Experimental Dataset. The model methodology proposed in this paper is oriented towards real-world scenarios, employing a cyber-physical-social integration approach [31] to construct an experimental system that incorporates data, models, and engines [32]. This paper uses real-world dataset Alibaba Cluster Data V2018, an 8-day workload trace from Alibaba Cloud, covering 4,000 servers with static and runtime metrics for about 9000 online tasks and 400,000 offline tasks. Key metrics include CPU, memory, and disk I/O, sourced from container

metadata (container_meta.csv) and resource usage (container_usage.csv) tables, detailing container attributes and resource consumption.

4.2 Performance on CPU Workload

A comparative evaluation of mean absolute error (MAE) among different models demonstrates that the method based on the LSTMFlash model achieves a higher prediction accuracy at the second level, while exhibiting marginal performance differences at minute-level and hour-level. As shown in Fig. 2, the proposed method yields an average improvement of 16.12% over the baseline methods and 15.18% over the Transformer models at the second-level, 12.63% and 6.27% at the minute-level, 14.04% and 12.51% at the hour-level.

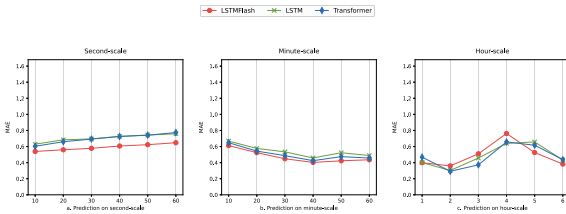


Fig. 2. Comparison of training performance on CPU workload

4.3 Performance on Memory Workload

Comparative evaluation shows the proposed model achieves optimal second-level prediction accuracy, with a 20.99% improvement over baseline and 16.64% over Transformer-based models as shown in Fig. 3. However, it exhibits mixed performance at minute- and hour-levels, the average MAE differences between the proposed method and the two benchmarks are 0.13 and 0.09, while at the hour-level scale, the differences narrow to 0.03 and 0.13, highlighting its scale-dependent adaptability and underscoring the need for granularity-aware optimization in workload forecasting frameworks.

4.4 Efficiency on CPU and Memory Workloads

As shown in Table 1, our comparative analysis of model convergence times reveals that the LSTMFlash model achieves the fastest convergence across most of the temporal granularity scales (second, minute, and hour scales), while the LSTM and Transformer exhibits prolonged training durations at the second level scale due to slower convergence rates and higher epoch requirements. Compared to the baseline LSTM, our method reduces the training time by 25.21% at second-level, 12.80% at minute-level, and 31.74% at hour-level, and achieves greater

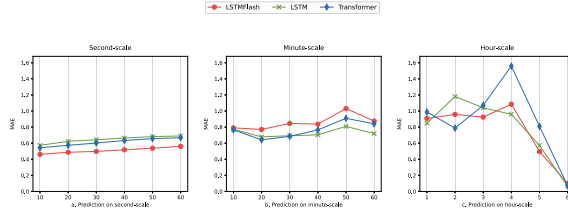


Fig. 3. Comparison of training performance on memory workload

efficiency gains over the Transformer baseline, with reductions of 66.21%, 2.06%, and 67.04% on the same scales in CPU workload workload, and yields an average improvement of 5.88% over the baseline methods and 64.5% over the Transformer models at the second-level, 1.52% and 31.59% at the minute-level, 5.86% and 29.57% at the hour-level. These results demonstrates its computational efficiency in time-sensitive training scenarios.

Table 1. Training Time Comparison on different resources and time scales

		20 s	40 s	60 s	20 min	40 min	60 min	2 h	4 h	6 h
CPU	LSTMFlash	4766.79	4619.97	4667.99	4780.74	4519.06	4335.03	1463.42	791.90	737.79
	LSTM	6141.48	5270.10	6108.18	5274.23	5031.40	5460.61	4614.21	2899.37	506.85
	Transformer	14178.58	13979.09	14397.25	4590.31	4549.90	4250.35	4700.92	3174.22	1735.11
Memory	LSTMFlash	2818.60	2847.21	2905.10	2968.00	2739.20	2701.96	2285.13	1597.44	832.49
	LSTM	3055.58	3064.42	2988.85	2921.42	2745.31	2872.43	2351.46	1581.49	988.48
	Transformer	14302.31	6682.00	6792.86	4206.47	4250.90	3846.95	3225.98	2208.54	1222.07

5 Conclusion

This paper introduces LSTMFlash, a workload prediction model for containerized environments, combining enhanced LSTM with flash attention to capture fine-grained and multiscale temporal dynamics in workloads. Experimental results demonstrate that LSTMFlash achieves significant reductions in training time and reduces mean absolute error (MAE) for most cases compared to LSTM and Transformer, which reveals its efficiency and performance. The model can offer a more effective and efficient approach to workload prediction for container elastic scheduling, thus improves the service reliability for CSPs. Our model demonstrates potential in workload prediction, but still needs to improve the performance and stability in complex scenarios, and our future work will focus on this goal and conduct more optimization.

References

1. Martin, J.P., Kandasamy, A., Chandrasekaran, K.: Exploring the support for high performance applications in the container runtime environment. *Hum.-cent. Comput. Inf. Sci.* **8**(1), 1–15 (2018). <https://doi.org/10.1186/s13673-017-0124-3>
2. Hussein, M.K., Mousa, M.H., Alqarni, M.A.: A placement architecture for a container as a service (CaaS) in a cloud environment. *J. Cloud Comput.* **8**, 1–15 (2019)
3. Bhardwaj, A., Krishna, C.R.: Virtualization in cloud computing: moving from hypervisor to containerization a survey. *Arab. J. Sci. Eng.* **46**(9), 8585–8601 (2021)
4. Hamzaoui, I., Duthil, B., Courboulay, V., Medromi, H.: A topical review on container-based cloud revolution: Multi-directional challenges, and future trends. *SN Comput. Sci.* **5**(4), 416 (2024)
5. Kang, D.K., Choi, G.B., Kim, S.H., Hwang, I.S., Youn, C.H.: Workload-aware resource management for energy efficient heterogeneous docker containers. In: 2016 IEEE Region 10 Conference (TENCON), pp. 2428–2431. IEEE (2016)
6. Zhu, K., et al.: Proactive hybrid autoscaling for container-based edge applications in kubernetes. In: Leung, V.C., Li, H., Hu, X., Ning, Z. (eds.) *Qshine 2023. Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering*, vol. 574, pp. 330–345. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-65123-6_24
7. Yadav, M.P., Rohit, Yadav, D.K.: Maintaining container sustainability through machine learning. *Clust. Comput.* **24**(4), 3725–3750 (2021)
8. Meng, Y., Rao, R., Zhang, X., Hong, P.: CRUPA: a container resource utilization prediction algorithm for auto-scaling based on time series analysis. In: 2016 International Conference on Progress in Informatics and Computing (PIC), pp. 468–472. IEEE (2016)
9. Dao, T., Fu, D., Ermon, S., Rudra, A., Ré, C.: FlashAttention: fast and memory-efficient exact attention with IO-awareness. *Adv. Neural. Inf. Process. Syst.* **35**, 16344–16359 (2022)
10. Alibaba cluster data v2018. <https://github.com/alibaba/clusterdata/tree/v2018/>. Accessed 14 Mar 2025
11. Xie, Y., et al.: Real-time prediction of docker container resource load based on a hybrid model of ARIMA and triple exponential smoothing. *IEEE Trans. Cloud Comput.* **10**(2), 1386–1401 (2020)
12. Messias, V.R., Estrella, J.C., Ehlers, R., Santana, M.J., Santana, R.C., Reiff-Marganiec, S.: Combining time series prediction models using genetic algorithm to autoscaling web applications hosted in the cloud infrastructure. *Neural Comput. Appl.* **27**, 2383–2406 (2016)
13. Kang, P., Lama, P.: Robust resource scaling of containerized microservices with probabilistic machine learning. In: 2020 IEEE/ACM 13th International Conference on Utility and Cloud Computing (UCC), pp. 122–131. IEEE (2020)
14. Ding, Z., Feng, B., Jiang, C.: COIN: a container workload prediction model focusing on common and individual changes in workloads. *IEEE Trans. Parallel Distrib. Syst.* **33**(12), 4738–4751 (2022)
15. Zhang, L., et al.: A novel hybrid model for docker container workload prediction. *IEEE Trans. Netw. Serv. Manage.* **20**(3), 2726–2743 (2023)
16. Ding, Z., Feng, B., Yu, W.: Work-in-progress-a large-scale workload forecasting model for containers. In: 2023 IEEE International Conference on Web Services (ICWS), pp. 725–727. IEEE (2023)

17. Rimal, B.P., Choi, E., Lumb, I.: A taxonomy and survey of cloud computing systems. In: 2009 Fifth International Joint Conference on INC, IMS and IDC, pp. 44–51. IEEE (2009)
18. Yadav, M.P., Gupta, S., Dileep, A.D., Gonsalves, T.A.: A joint feature selection framework for multivariate resource usage prediction in cloud servers using stability and prediction performance. *J. Supercomput.* **74**(11), 6033–6068 (2018). <https://doi.org/10.1007/s11227-018-2510-7>
19. Shyam, G.K., Manvi, S.S.: Virtual resource prediction in cloud environment: a Bayesian approach. *J. Netw. Comput. Appl.* **65**, 144–154 (2016)
20. Google clusterdata 2019 traces. <https://github.com/google/cluster-data>. Accessed 14 Mar 2025
21. Lu, C., Ye, K., Xu, G., Xu, C.Z., Bai, T.: Imbalance in the cloud: an analysis on Alibaba cluster trace. In: 2017 IEEE International Conference on Big Data (Big Data), pp. 2884–2892. IEEE (2017)
22. Cheng, Y., Chai, Z., Anwar, A.: Characterizing co-located datacenter workloads: an Alibaba case study. In: Proceedings of the 9th Asia-Pacific Workshop on Systems, pp. 1–3 (2018)
23. Bao, L., Yang, J., Zhang, Z., Liu, W., Chen, J., Wu, C.: On accurate prediction of cloud workloads with adaptive pattern mining. *J. Supercomput.* **79**(1), 160–187 (2023)
24. Zhang, T., Ramakrishnan, R., Livny, M.: BIRCH: an efficient data clustering method for very large databases. *ACM SIGMOD Rec.* **25**(2), 103–114 (1996)
25. Hochreiter, S., Schmidhuber, J.: Long short-term memory. *Neural Comput.* **9**(8), 1735–1780 (1997)
26. Rumelhart, D.E., Hinton, G.E., Williams, R.J.: Learning representations by back-propagating errors. *Nature* **323**(6088), 533–536 (1986)
27. Vaswani, A., et al.: Attention is all you need. In: Advances in Neural Information Processing Systems, vol. 30 (2017)
28. Lim, B., Arik, S.Ö., Loeff, N., Pfister, T.: Temporal fusion transformers for interpretable multi-horizon time series forecasting. *Int. J. Forecast.* **37**(4), 1748–1764 (2021)
29. Arbat, S., Jayakumar, V.K., Lee, J., Wang, W., Kim, I.K.: Wasserstein adversarial transformer for cloud workload prediction. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 36, pp. 12433–12439 (2022)
30. Tay, Y., Dehghani, M., Bahri, D., Metzler, D.: Efficient transformers: a survey. *ACM Comput. Surv.* **55**(6), 1–28 (2022)
31. Xue, X., et al.: Computational experiments for complex social systems: experiment design and generative explanation. *IEEE/CAA J. Autom. Sinica* **11**(4), 1022–1038 (2024)
32. Xue, X., et al.: Computational experiments for complex social systems: integrated design of experiment system. *IEEE/CAA J. Autom. Sinica* **11**(5), 1175–1189 (2024)